

POO surgiu nos anos 60 com
Alan Kay considerando um
dos pilares da orientação a
objeto.

- O principal elemento da
POO é o objeto!!!

- Objeto: Um objeto é
responsável por realizar tarefas
específicas.

- É por meio de interações
entre objetos que uma tarefa
computacional é realizada.

- Qualquer coisa é um objeto.

- Objetos realizam tarefas por meio da requisição de serviços a outros objetos.

- A interação entre objetos é que constitui todo o sistema.

• Java:

- A nixtose bee' barade

em C.

- Pibrente da C beza

limpeza da memória de
maneira automaticamente.

- Convenção para

ceder em Java: Camel Case

para classes, atributos e métodos.

Tipo das classes:
pública, privada.
não tem o tipo. Uma classe é
com tipo

Métodos:
main: classe onde irá iniciar o
programa!

permite static no main
(String args[]) { ... }

↳ Em Java, collage é
designtar uma classe
main para o início da execução.

- A JVM (máquina virtual de
java) só executa bytecode.

Exemplo: Uma biblioteca de
Java.

- Objeto / instância

- Armazena dados

- Tem estado interno

- Tem comportamento

- Incapacidade de valores!

- "Deixar os dados

- privados, sem alterar

- diretamente, apenas o

próprio objeto se altera!"

• Criando classe:

perde-se uma Referência
↳ Criando a Tipo Referência

3

public class MatrixWorld {
public static void main
(String args[]) {

Chama o
construtor padrão
para lidar com o dado

na
memória

variáveis

memória

Refere-se a 2d2;

2d2 -> new Referece();

↳ Criando objeto

new é usado em POO

para criar objetos

ao criar o objeto começa

a alocação na memória.

variável
de referência

não é a variável
apenas a referência
como um
ponteiro
em C!

Exemplos: Operador "new" cria um objeto.

Exemplos: Declarações não cria referências, em POO quem faz alguma coisa é o objeto.

Exemplos: Quando apenas declaramos uma variável e não damos "new" então está declarando e NULL

Em Java ele não utiliza o construtor padrão de null não cria nenhum construtor.

Exemplos Java Robot E
não cria um construtor automaticamente }

• Tipos primitivos em Java:

2 de sistema binário

0 ou 1

- Inteiros

- byte $\rightarrow$ 8 bits

$\rightarrow$ 8² possibilidades

- short $\rightarrow$ 16 bits

$\rightarrow$ 16² possibilidades

- int $\rightarrow$ 32 bits

$\rightarrow$ 32² possibilidades

- long $\rightarrow$ 64 bits

$\rightarrow$ 64² possibilidades

- Real flutuante (Decimais)

- float $\rightarrow$ Precisão de 7 casas decimais.

- double $\rightarrow$ Precisão de 15 casas decimais.

- tipo lógico

- boolean $\rightarrow$ 8 bits

• String (é uma classe que é um tipo)

- String

ASCII - 8 Bits $\rightarrow 8^2$ possibilidades de caracteres.

UNICODE 16 Bits $\rightarrow 16^2$ possibilidades de caracteres.

- O Unicode difere da ASCII

porque de melhor forma abarca todos os idiomas mundiais.

* O Java usa unicode e ocupa 16 bits por caractere.

ALERTA: Em padrão e Java
em tipos primitivos quando o
mínimo é constante, então se
declararmos float ou int temos
que "arredondar" a linguagem ao
fazer uma atribuição. Por isso Java
é fortemente tipada então é
um tipo de variável sempre será
o mesmo!

Exemplo: float x;
x = 7.72;

32 bits
constante então
precisão estável, logo
deará erro.

O correto: x = 7.72f

Precisamos
dizer que é float
por isso Java é fortemente tipada.

OBS: Para armazenar as
variáveis dos objetos usamos
ponto ".".

Exemplo:

Acervo
↓
System.out.println()

Acervo
↓

Característica: Padrão, o que é
um padrão? Padrão é uma
solução ideal e constantemente
utilizada para solucionar
um problema recorrente!

* Criar uma classe;

Precheck results

Vale 5.0 ponto(s).

☐ Marcar questão

Criar uma classe Livro em Java, sabendo que qualquer livro possui um atributo booleano novo, que indica se o livro é novo ou usado. Escolha bem os tipos de dados para cada um dos atributos.

For example:

Test	Result
<pre>Livro livro = new Livro(); System.out.println(livro.novo);</pre>	false

Answer: (penalty regime: 0%)

```
1 public class Livro {
2     String titulo;
3     String autoria;
4     String categoria;
5     String editora;
6     int numeroDePaginas;
7     int anoDeLancamento;
8     boolean novo;
9 }
10
11
```

Precheck

Verificar

Precheck only

Test	Expected	Got

↳ Fica pelo AME2, o
que pede o nome da
mãe e exatamente como
ele quer.

Inesperadamente disse
os nomes privados para
que ninguém tenha acesso
sem um objeto.

```

public void setNome (String nome) {
    if (nome != null) {
        this.nome = nome;
    }
}

```

↳ Para retornar um valor

```

public void setNivelBateria (int nivelBateria) {
    if (nivelBateria >= 0 || nivelBateria <= 100) {
        this.nivelBateria = nivelBateria;
    }
}

public int getNivelBateria () {
    return nivelBateria;
}

```

• Mixture:

- O que é uma mixture?

Mixture é como se fosse uma
bomba, mas que pertence a alguns
objetos, a mixture está escrita
dentro da classe, quando criamos
um objeto usando o tipo
classe os métodos não são bombeiros
responsáveis por alterar os
valores dentro dos objetos.

• Construtores:

- O que é um construtor:

- É um método especial que não retorna nada (nem nada!).

- Serve apenas para criar e inicializar o objeto!

- O que um construtor faz de

principal:

- Inicializa atributos
(com valores iniciais)

- Garantias invariantes.
(validades obrigatórias)

- Preparar dependências:
(instâncias de outros
objetos que a classe precisa
para funcionar)

Quando criamos um objeto:

Pessoa p = new Pessoa ("Maria");

↳ Após criarmos um objeto o mesmo precisa ser iniciado, precisamos passar para ele seus "atributos" iniciais, por isso dizemos que o construtor de fato "constrói" o objeto.

Agora devemos criar um construtor, cada linguagem tem suas regras, mas em Java o C++ o construtor deve ter o mesmo nome da classe!

* Lembrando: Se não criarmos um construtor a linguagem cria o "construtor padrão" que é um pouco limitado, na maioria dos casos é melhor criarmos um!

java

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        if (idade < 0) {  
            throw new IllegalArgumentException("Idade inválida");  
        }  
        this.nome = nome;  
        this.idade = idade;  
    }  
}
```

Percutamos que a classe e o construtor tem o mesmo nome.

Nome

→ Construtor

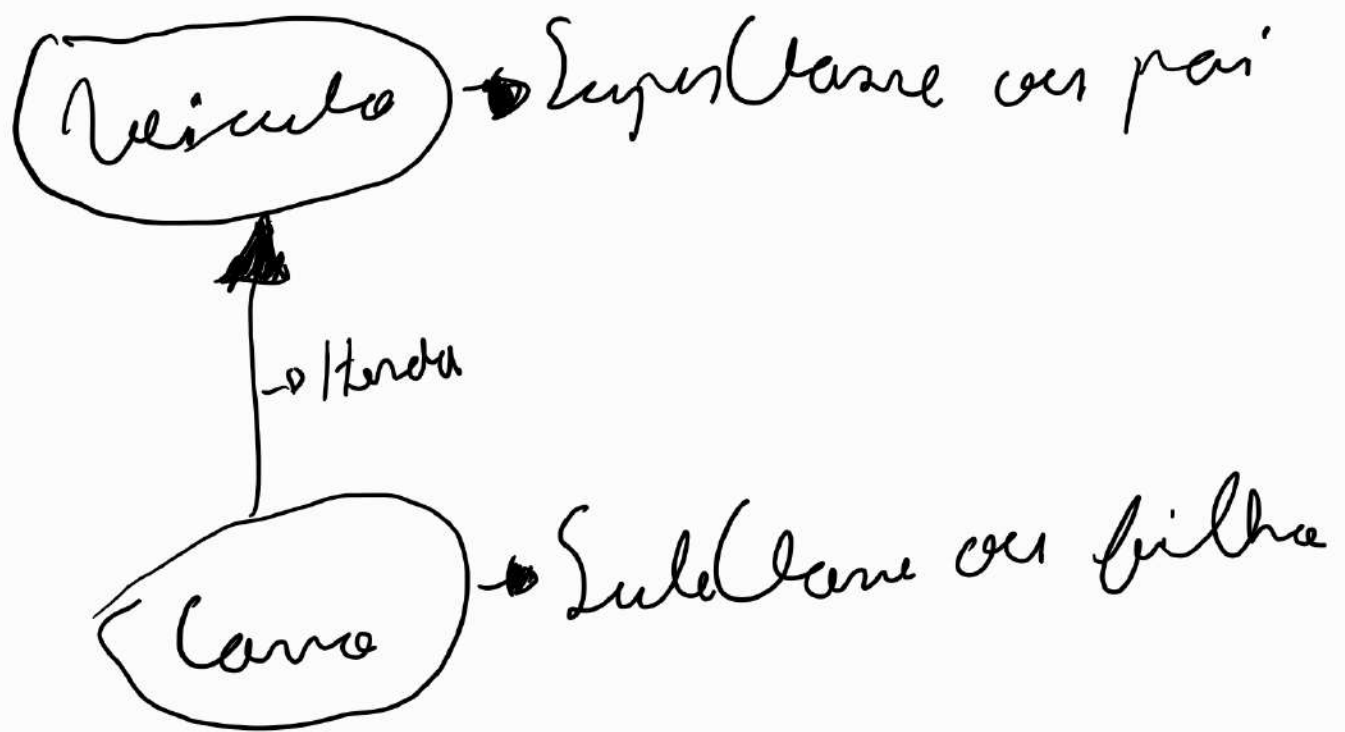
OBS: O construtor precisa ter o mesmo nome que a classe, pois o compilador logo após a criação de um novo objeto de classe X vai em busca do construtor de mesmo nome para iniciar o objeto.

OBS-2: O construtor é sempre definido dentro da classe, porque ele faz parte da própria definição da classe.

• Herança ou extends:

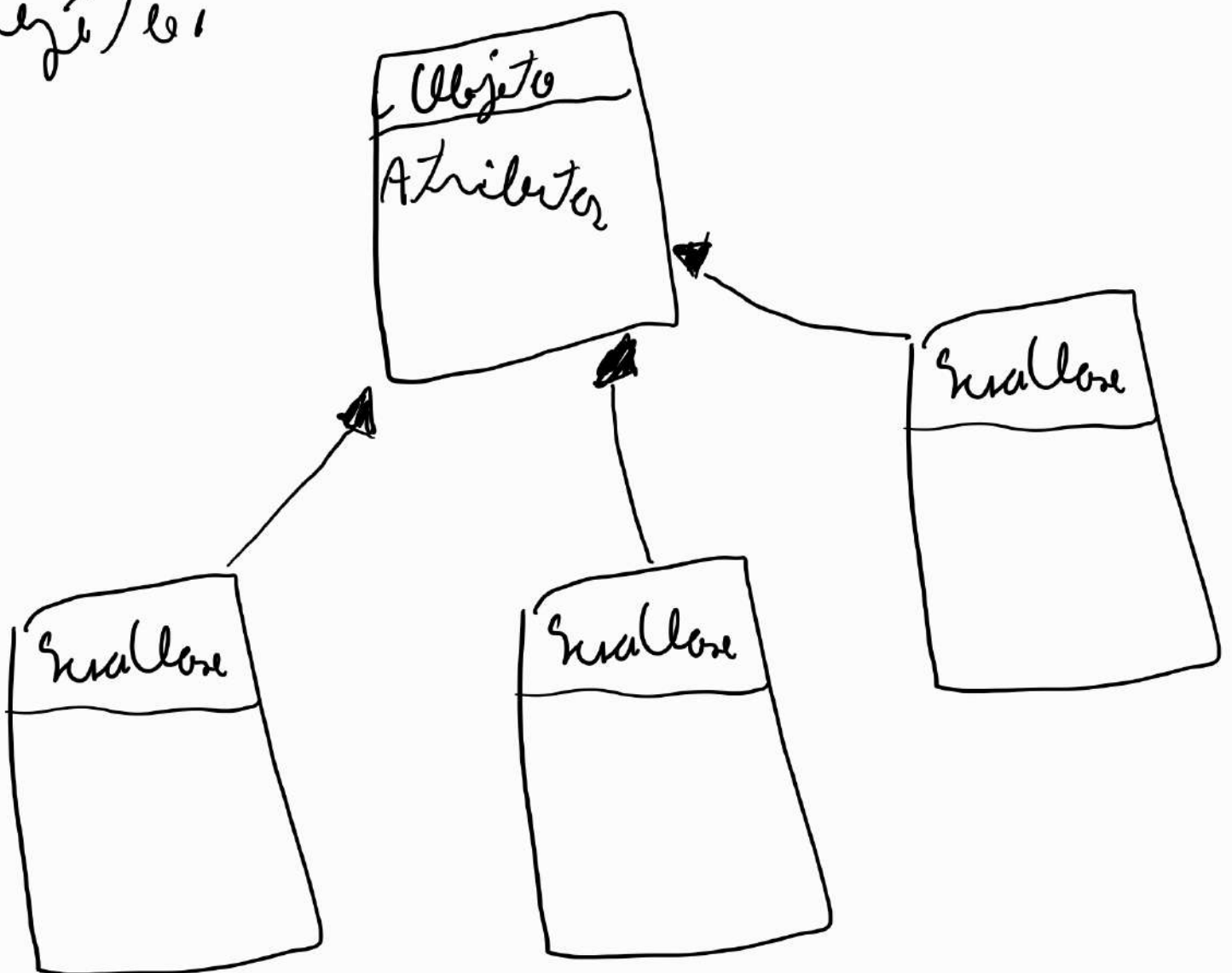
```
public class Veiculo {  
    private String cor;  
  
    public Veiculo(String cor) {  
        this.cor = cor;  
    }  
  
    public String getCor() {  
        return this.cor;  
    }  
}  
  
public class Carro extends Veiculo {  
    private int numeroDePortas;  
  
    public Carro(String cor, int numeroDePortas) {  
        super(cor);  
        this.numeroDePortas = numeroDePortas;  
    }  
  
    public int getNumeroDePortas() {  
        return this.numeroDePortas;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        Carro carro = new Carro( cor: "Vermelho", numeroDePortas: 4);  
        System.out.print(carro.getCor() + " ");  
        System.out.print(carro.getNumeroDePortas());  
    }  
}
```

Carro é "filho" de Veiculo por isso herda seus atributos e métodos.

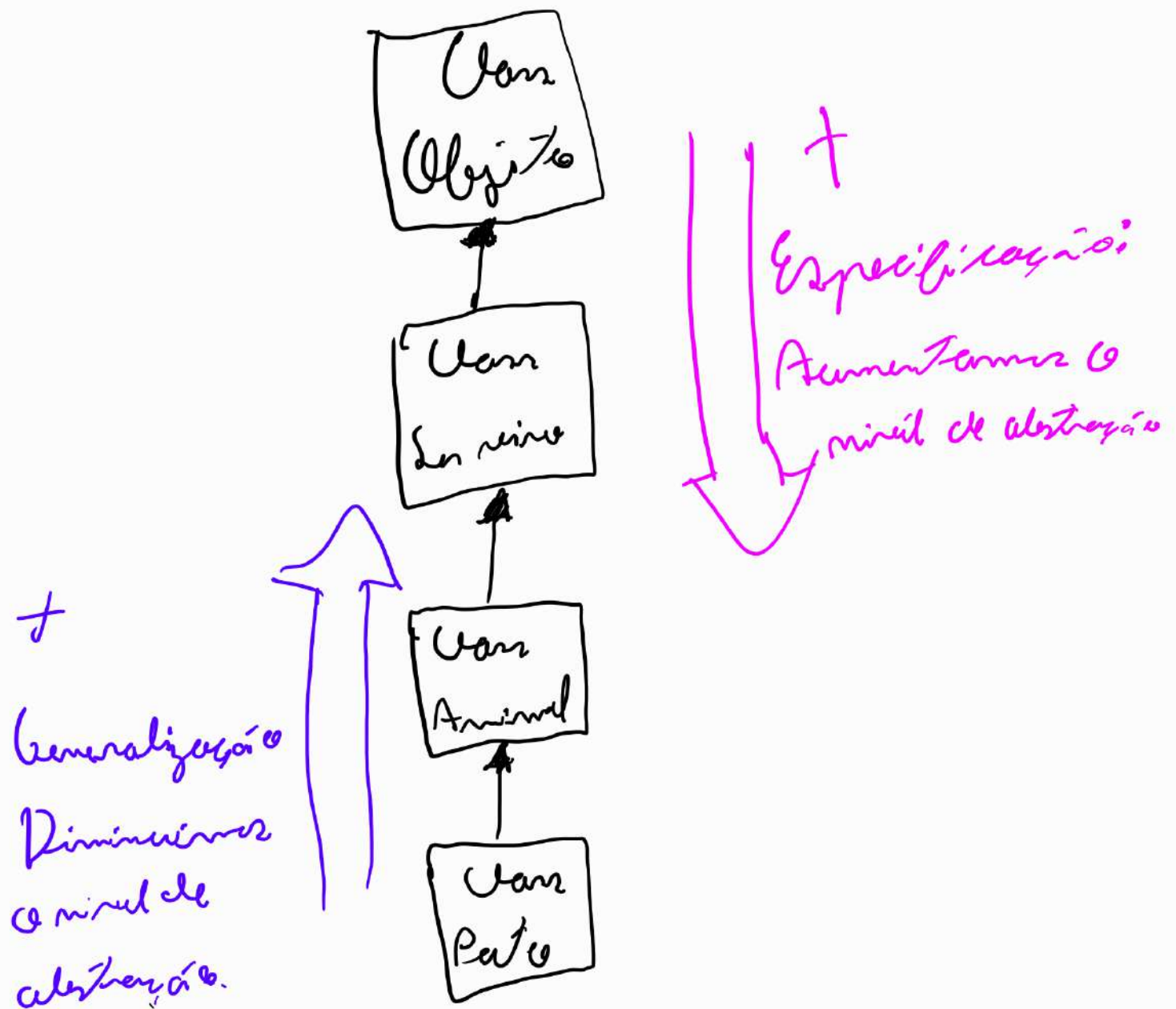


- Basicamente a subclasse pode usar atributos e métodos próprios e protected da super classe, porém o contrário não pode acontecer! A filha herda do pai e mãe o pai da filha.
- Há também a possibilidade de herança múltipla quando uma subclasse herda de mais de uma superclasse.

• Em Java temos uma classe
"superclasse", a classe: Objeto
então todas as classes que
você criar herdam a classe
Objeto!



• Ao fazermos heranças vamos aumentando o nível de especificação



• Em Java não temos classes com herança múltipla, em Java temos apenas herança simples!

• Verificação de atribuição de cores:

Se for de mesmo tipo da corte:

Se for de um tipo x e receber

a subclasse de x inteira da corte.

Todos centros peritilizados são
vão das cores!

• Uma rede tem peritilidade proibida
de de peritilidade!

• **Objeto:**

- Tipo agregado de dados é um tipo de dados definido pelo programador!



Atributos: diz respeito aos valores que um objeto tem, os dados que ele guarda. Tem ligação direta com a definição/aplicação de tipos de dados

Comportamentos: diz respeito às ações que um objeto pode fazer. Tem ligação direta com a definição/aplicação de métodos

• Exemplos de alguns atributos

- Cada tipo primitivo tem um tamanho definido, como pode ser visto na tabela abaixo:

Tipo	Tamanho (bits)	Tipo	Tamanho (bits)
boolean	1	char	16
byte	8	short	16
int*	32	long	64
float	32	double**	64

* padrão para constantes inteiras;

** padrão para constantes de ponto flutuante.

- Como podemos acessar as variáveis de instância de um objeto?

- Acessamos por meio de operadores "."

```
public class Robopolis{  
    public static void main(String[] args){  
        Robot r2d2 = Instanciação new Robot(); Constructor  
        r2d2.totalSensores = 2;  
        r2d2.nivelCarga = 10; Atribuindo  
        System.out.println(r2d2.totalSensores);  
        System.out.println(r2d2.nivelCarga);  
        System.out.println(r2d2.isCarregando);  
    }  
}
```

^{Uso}

• Classes abstratas:

- Nem todos os classes são prontas de serem instanciadas:
- Algumas classes existem simplesmente para agrupar comportamentos similares:
- A estratégia de usar classes abstratas é ter uma superclasse que dita o comportamento, enquanto os subclasses concretos têm a liberdade de prover a implementação que acharem relevante!

- Usar Abstrator não muito inteiro no reuse, incrementando o mecanismo de polimorfismo.

- Para criar uma classe abstrata devemos usar o modificador "abstract" em sua definição.

- Métodos abstratos:

- Não tem implementação.

- As subclasses de uma classe que possui métodos abstratos devem implementar os métodos abstratos herdados da superclasse, caso contrário não poderão ser instanciadas!

- Um método abstrato é uma modificação "abstract" e possui apenas uma assinatura seguida de ";"

Exemplos:

```
abstract public void correr();  
abstract int getNumeroDePernas();  
Peruano apenas a assinatura;
```

- Alguns membros não podem ser declarados como abstratos:

- Construtores
- Métodos estáticos.
- Métodos privados.

• Sobrecarga de métodos:

↳ (Overloading)

- Importante para a polimorfismo porque reutiliza um método, com os parâmetros diferentes ou tipos diferentes!

- Cada método tem uma assinatura!

- Assinatura: Nome e parâmetros

↳ A assinatura se diferencia pela ordem dos parâmetros ou de tipos diferentes!

Obs: Sobrecarga só acontece na mesma classe!

- Construtores:

- São "métodos" especiais

Chamados no momento da criação do objeto.

↳ Só é chamado no momento da criação de um objeto! NÃO PODE SER CHAMADO DEPOIS!

↳ Construtores também podem ser sobrecarregados!

É muito importante lembrar
que o construtor SEMPRE deve
analisar se os dados passados
fazem sentido, antes de armazenar
dentro do objeto!

This → Referência

e

this()

→ Construtor que chama construtor!

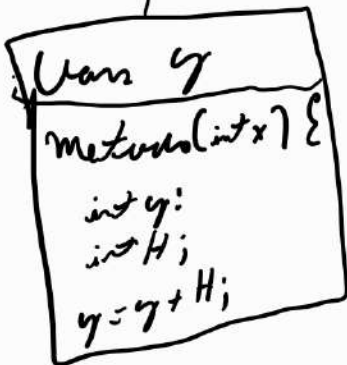
É útil para corrigir erros em objetos
passando alguns dados de novo, uma maneira opcional!

- this() é a única maneira de descrever
na hora de construir um objeto! Porque se temer
um parâmetro é obrigatório passarmos um valor
para ele!

Redefinição de Métodos:

- Cópia o método com exatamente a mesma assinatura, mas o código diferente.

- Redefinição funciona só em outra classe! Que está herdando uma classe!



Método com exatamente a mesma assinatura, mas códigos diferentes! Porém "mesmo" método deve manter a mesma semântica (Sentido) deve ter o mesmo objetivo dele.

- Unary operators:
 - (+ variable) → torna o número positivo
 - (- variable) → torna o número negativo

- Referência ao método Super:

- Revisão para prova:

- Um objeto é uma entidade que possui um **estado** e um **conjunto de ações definidas para funcionar nesse estado**.

- O estado é representado como o valor de um conjunto de atributos de objetos.

- Quando criamos um objeto apartir de uma classe estamos **instanciando** um objeto.

- Pilares de POO: Abstração, Herança, Polimorfismo, Encapsulamento.

- Uma instância de uma subclasse é uma instância **de todas** as suas superclasses.

- Encapsulamento: Qualquer mecanismo que nos permita "esconder" a implementação de um objeto.

- **Private** → Acesso na mesma classe.

- **Default** → Acesso na mesma classe.

- **Protected** → Acesso de outras subclasses.

- **Public** → Qualquer classe tem acesso.

- Polimorfismo capacidade de tratar uniformemente uma coleção de objetos, emitir e receber a mesma mensagem e responder de maneira diferente.

- Execução do Java: Java $\xrightarrow{\text{Compilador}}$ V. Java $\xrightarrow{\text{Interpretador (JVM)}}$ Sistema
↳ Bytecode

- Java é uma linguagem com tipagem forte!

- JVM = interpretador, javac = compilador

- JRE = Ambiente executável, JDK = Ambiente de produção

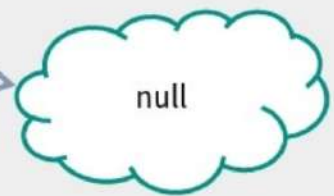
- O operador "new" é "quem cria o objeto" criando o endereço dele na memória ram

- Quando criamos uma variável com o tipo do objeto estamos apenas fazendo uma referência a ele!

Robot r2d2;



Referência (tipo em pontos)

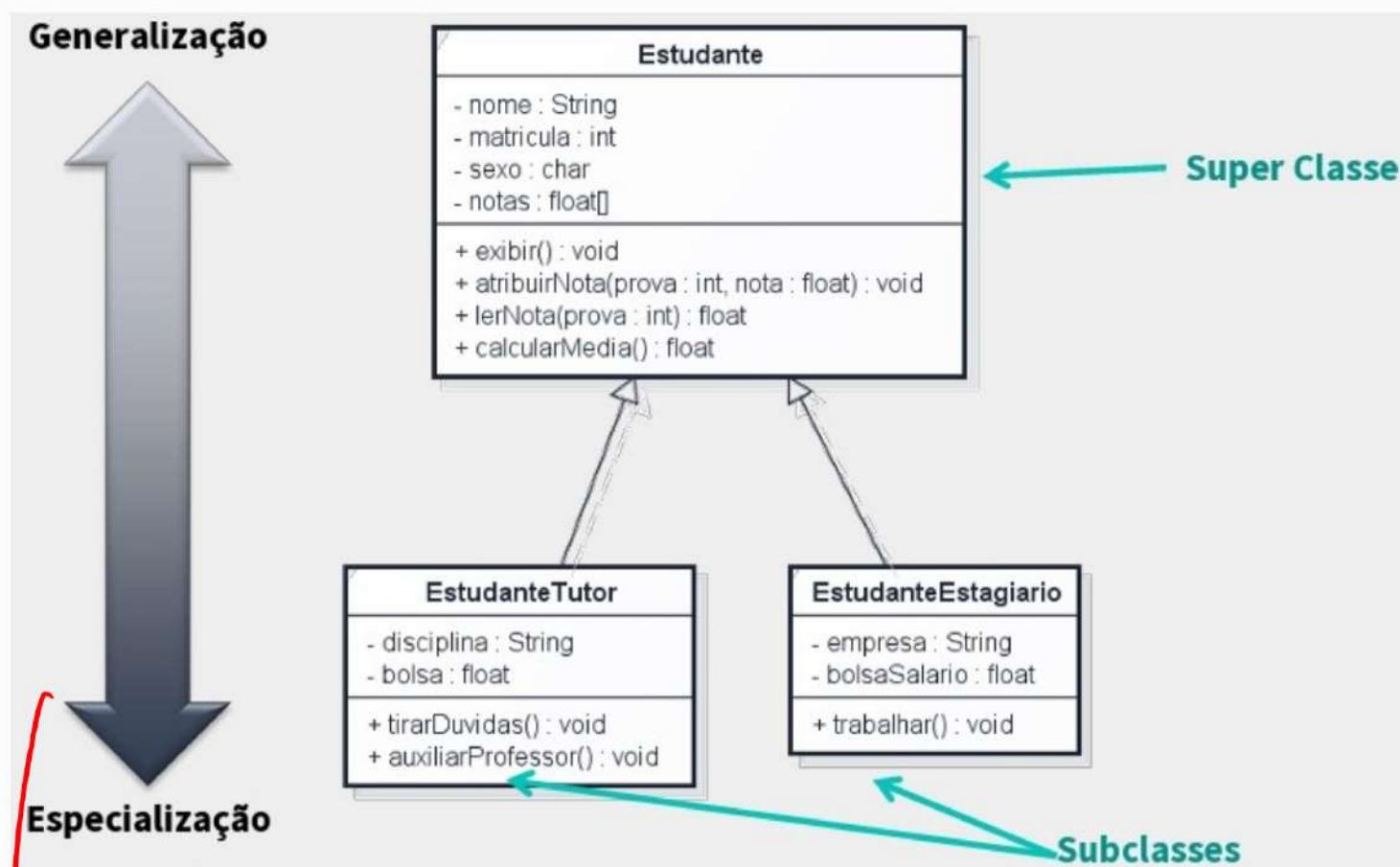


r2d2 = new Robot();

↳ De fato está criando o objeto na memória.



- Uma subclasse deve apresentar novas comportamentos ou propriedades, ou até mesmo mudar o comportamento herdado



Quanto mais subclasses mais devemos especializar a abstração.

- Java.lang.Object é a mãe de todas as classes!
Uma classe sem extends em seu cabeçalho sempre será subclasse de Java mãe Object!

- A linguagem Java só tem herança simples! Ou seja uma subclasse herda apenas uma superclasse!

NO MECANISMO DE HERANÇA...

- Os atributos e métodos privados são herdados, **mas não podem ser acessados diretamente**. Deve-se usar métodos acessadores (*getters*) e modificadores (*setters*)
- Uma instância de uma subclasse é uma instância de todas as suas superclasses
- Objetos da subclasse comportam-se como objetos da superclasse
- Objetos da subclasse podem ser usados no lugar de objetos da superclasse

↳ Mecanismos importantes:

- Sobrecarga de métodos:
 - Métodos com os mesmos nomes, mas assinaturas diferentes, mantendo também a mesma lógica do objetivo final.
 - Para herdarmos a assinatura de um método os parâmetros têm que diferir em tipo, ordem ou número.
 - Métodos sobrecarregados não precisam ter o mesmo tipo de retorno!

• Construtores:

- São "métodos" especiais chamados no momento da criação de um objeto (new!).

- Servem para setar o estado inicial de um objeto quando ele é criado (um construtor só é chamado quando o objeto é criado!).

- Pode haver mais de um construtor por classe (sobrecarga).

```
public class Data {  
    int dia, mes, ano;  
  
    public Data() {  
        ano = 2010; }  
    ...  
}
```

(O "()" está chamando

Utilizando... o construtor padrão!

...

```
Data hoje = new Data();
```

- Quando dois construtores (overloading):

```
public class Data {
```

```
...
```

```
public Data() {  
    ano = 2004; } ←
```

```
public Data( int numAnos ) {  
    ano = numAnos; } ←
```

```
...
```

```
}
```

Utilizando...

```
...
```

```
Data hoje = new Data();
```

```
Data depois = new Data( 2005 );
```

Chamando o
padrão

Chamando construtor
em específico.

Percebemos que ao adicionar
um número inteiro o interpretador
vai procurar pelo construtor que
tenha aquela assinatura para instanciar o
objeto.

OBS: Construtores podem ser sobrecarregados
apenas na mesma classe!!!

A chamada do construtor **this ()** pode ser usada para encadear construtores sobrecarregados (**não confundir com a referência this**)

- A chamada **this ()** só pode ser usada em definições de construtores, e, quando usada, **deve sempre ser a primeira sentença no código do construtor**

```
public Estudante(String nome, char  
sexo) {  
    this.sexo = sexo;  
    this.nome = nome; }
```

```
public Estudante(String nome, char  
sexo, int matricula) {  
    this( nome, sexo );  
    this.matricula = matricula; }
```

↳ this pode ser usado para construir com mesmo objeto usando dois construtores diferentes.

• Revisão para prova prática:

- Em cada arquivo .java conseguimos ter apenas uma classe principal com o encapsulamento público! os outros classes devem estar dentro de tipos de encapsulamento de convenientemente e default (não escreva nenhum). A classe principal deve sempre ter o mesmo nome do arquivo, java!

- O compilador vai iniciar a execução do código a partir da classe que tiver o método main:

```
public class Exercicio01 {  
    Run | Debug  
    public static void main(String[] args) { //Metodo main  
  
        Pessoa p = new Pessoa(nome: "Maria", idade: 20);  
        System.out.println(p.exibirDados());  
    }  
}
```

↳ Por onde o interpretador começa a iniciar o código!

- Setters e getters:
- Dois métodos muito importantes para o encapsulamento!

- O set serve para proteger que um dado não seja atribuído de qualquer maneira errada. Assim, antes de atribuir, ele faz uma validação para saber se aquele dado faz sentido de ser não em um atribuído.

Exemplo:

Se o set poderíamos atribuir diretamente -70 dentro de um atributo idade, sendo que não a temos qualquer coisa ter a idade de -70!

Então o set através de uma validação não deixaria esse dado passar e armazenar no atributo!!!

```
public class Exercicio01 {
```

Run | Debug

```
public static void main(String[] args){ //Metodo main  
  
    Pessoa p = new Pessoa(nome: "Maria", idade: 20);  
    p.idade = -10;  
    System.out.println(p.exibirDados());  
}
```

↳ Alteramos diretamente o atributo idade de objeto p e passamos o valor que quisermos sem proteção alguma! (Como porque o atributo não está encapsulado!)

```
public class Exercicio02 {
```

Run | Debug

```
public static void main(String[] args){ //Metodo main  
  
    Pessoa p = new Pessoa(nome: "Maria", idade: 20);  
    p.setIdade(idade: 20);  
    p.setNome(nome: "Joao");  
    System.out.println(p.getIdade());  
    System.out.println(p.getNome());  
}
```

↳ Com os atributos com encapsulamento "private" não podemos alterá-los através de gets e sets, dessa maneira o que é passado pode ser validado antes de ser atribuído!

```

class Retangulo{

    int largura;
    int altura;

    public Retangulo(int largura, int altura){
        this.largura = largura;
        this.altura = altura;
    }

    public Retangulo(int x){
        this.largura = x;
        this.altura = x;
    }

    public int calcularArea(){
        return this.largura * this.altura;
    }

}

public class Exercicio03{

    Run | Debug
    public static void main(String[] args){
        Retangulo r = new Retangulo(x: 10);
        System.out.println(r.calcularArea());
        Retangulo h = new Retangulo(largura: 5, altura: 10);
        System.out.println(h.calcularArea());
    }
}

```

Seleção de construtores, o mesmo nome, mas com assinatura diferente! Para isso podemos construir dois objetos diferentes de acordo com os dados que precisamos!

Conseguimos criar dois objetos diferentes dependendo da assinatura que utilizarmos!

• Características:

- São classes que nunca
terão instâncias, servem
apenas como modelo
para outros. Muito úteis
para o polimorfismo!

- A estratégia de uma classe
abstrata é ter uma subclasses
que dita o comportamento, enquanto
as subclasses concretas têm a
liberdade de prever a implementação
que acharem relevante!

- Classes abstratas usam
o modificador abstract
para sinalizar que não
devem ser instanciadas!

* Métodos abstratos:

- Também devem ser declarados
como abstract.

- Não tem implementação,
apenas um protótipo.

- Uma classe que possui
um método abstrato não
pode ser instanciada!

- As subclasses que herdam
os métodos abstratos,
devem implementá-los!
Ou também não vão
instanciar!

PROPRIEDADES DAS CLASSES ABSTRATAS

1. Um classe abstrata **não** pode ser instanciada
2. É especificada com a palavra-chave **abstract**
3. Uma classe que possui algum método abstrato **deve** ser declarada abstrata
4. A subclasse de uma classe abstrata **deve** prover a implementação dos métodos abstratos herdados, caso contrário ela também será uma abstrata
5. Uma classe pode ser declarada abstrata mesmo que não possua métodos abstratos. Isso evita que se instanciem objetos dessa classe



• Interfaces:

- Define um contrato para as classes que precisam cumprir requisitos por uma classe!
- Pode conter restrições, assinaturas de métodos, métodos padrão, métodos estáticos e tipos.
- Uma interface não pode ser instanciada e nem conter construtores.
- Por padrão todos os métodos de uma interface são abstratos, ou seja, não possuem implementação no corpo!
- As interfaces herdam herança múltipla, então uma classe pode implementar várias interfaces!

• Classes abstratas VS Interfaces:

- Uma interface define um contrato de que fazes.

interface = O que fazes.

- Uma classe abstrata define a base de como fazes.

classe abstrata = como fazes.

• Usamos uma classe abstrata quando os comportamentos podem ser comuns entre "pai" e "filho".
evitando desperdiçar o espaço de código repetitivo!

- Exemplos:

- interface:

- Pagamento()

- Nadador()

- Vooar()

- Exibir()

- classe abstrata:

- Enciclopedia()

- RG()

- Desembarcar()

- Raridade()

- Data()

- Validar()

- Visitar()

- Verbo()

- metad()

• Um tem comportamentos gerais e outra funções específicas de classe.

• Enum:

- São um tipo especial de dado em Java.
- Permite que uma variável seja configurada com constantes predefinidas!

```
public enum Naipes {  
    ESPADAS(1),  
    OUROS(2),  
    PAUS(3),  
    COPAS(4);  
    public int valorNaipes;  
    Naipes(int valor) {  
        valorNaipes = valor;  
    }  
}
```

Handwritten notes on the code:

- Naipes* (green): Tipo Naipes
- 1* (yellow): Dado
- Red bracket on the left:* Possibilidades do Naipes

Recharamos enum com esta sintaxe como se fosse criar um classe, podemos declarar dentro ou fora de uma classe!

Enum cria um tipo de dado que pode conter varios valores, mas sempre constantes!

- Usar Enums precisando de construtores:

```
public enum Naipe {  
    ESPADAS(1, "Espadas"),  
    OUROS(2, "Ouros"),  
    PAUS(3, "Paus"),  
    COPAS(4, "Copas");  
    public int valor;  
    public String nome;  
    Naipe(int valor, String nome) {  
        this.valor = valor;  
        this.nome = nome;  
    }  
}
```

Construtores

- Exemplos:

```
public static void main(String[] args){  
    Naipe copas = Naipe.COPAS; tipo Naipe opcao COPAS!  
    Naipe paus = Naipe.PAUS;  
    System.out.println("Nome do Naipe = " + copas.nome); Acessando o dado  
    System.out.println("Valor do Naipe = " + copas.valor);  
    System.out.println("Nome do Naipe = " + paus.name());  
    System.out.println("Valor do Naipe = " + paus.valor);  
}
```

↳ Usar Enums quando precisarmos de constantes que podem assumir diferentes valores!

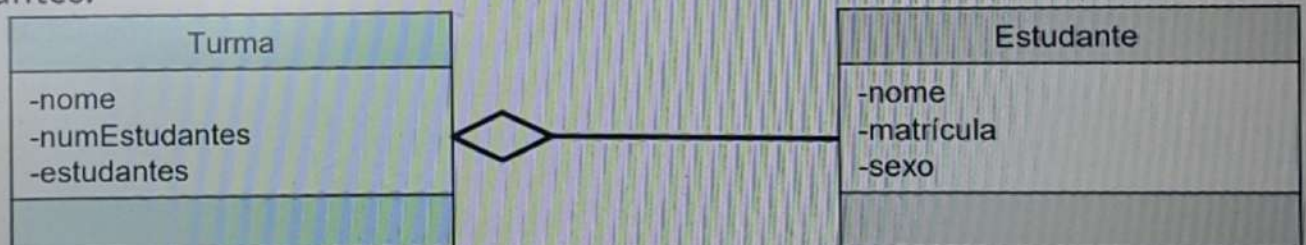
• Agregação:

- A agregação é um mecanismo fundamental de reuso de código.
- A agregação define uma relação do tipo "Tem um"

Exemplos:

- Uma turma é composta de estudantes
- Uma empresa é composta de departamentos
- Um aparelho é composto de peças.

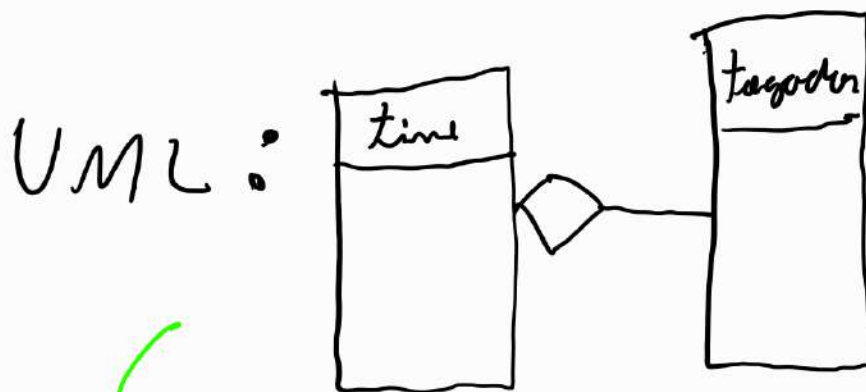
- Um objeto agregado é criado a partir de outros objetos constituintes, que são suas partes
- Veja abaixo a representação em UML para uma turma que é uma agregação de estudantes:



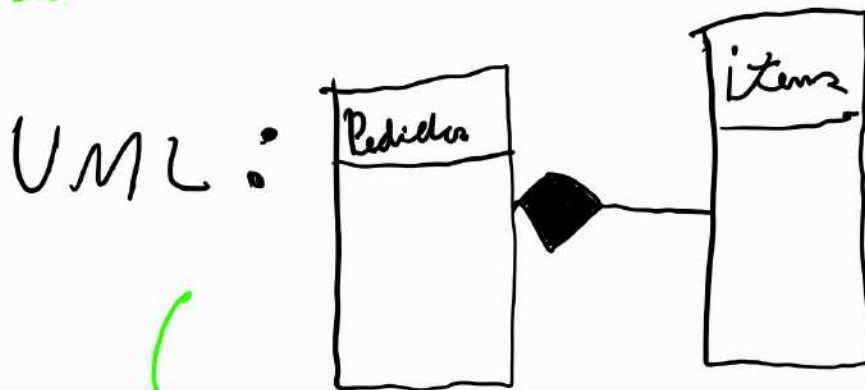
- **Código para a agregação Turma-Estudante:**

```
public class Turma {  
    String nome;  
    int numEstudantes;  
    Estudante estudantes[];  
    ... }  
}
```

↳ Um objeto em Java não podem conter outros objetos. Eles podem ter referências a outros objetos!



↳ Se apagarmos o time o jogador ainda continuará "vivo"



↳ Se apagarmos o pedido o item também será excluído!

Diagrama de classes:

Associação unidirecional
Vínculo entre duas classes

→ Relação fixa com outra classe

— Associação bidirecional

→ Herança/Generalização
extends

Associação
Informação de um objeto
precisa ser complementada
por um objeto de outra classe.

Composição
Vínculo mais forte entre
objeto e objeto - parte têm
que pertencer ao objeto - todo.

Dependência
Vínculo fraco entre duas classes
Relação momentânea entre classes
(usa um método ou atributo de uma em quando)

--- Interface (Realização)

• Arrays em Java:

- Um array é estrutura de dados que define uma coleção ordenada de um número fixo de elementos de dados homogêneos (mesmo tipo)

Em Java arrays não se criam, portanto precisamos criá-los antes de começarmos a usá-los.

- Declarando array unidimensional:

Não precisamos nem mesmo tamanho!
Tipo[] nome ou Tipo nome[]

- Existe para criar o array:

Aqui definimos o tamanho
nome = new Tipo[tamanho]

Importante: Em Java o tamanho do array não é fornecido na declaração, somente na criação!

IMPORTANTE:

- Normalmente já fazemos a declaração e a criação de uma vez:

Linha nome[] = new tipo[tamanho];

Exemplo:

[float motor[] = new float[5];]

Previamente fazer isso para em
toda array não deixar!



OBS: Quando acabamos de criar um array ele vem com valores default do seu tipo de dados!

Exemplos: int é 0, boolean é false, String é null

• Arrays Multidimensionais:

- Em Java podemos definir um array com muitos dimensões!

- Quantos mais o número de dimensões mais objetos serão criados!

- Podemos definir arrays multidimensionais conforme a sintaxe:

`tipo[] [] [] ... [] nome;`

ou

`tipo nome[] [] [] ... [];`

Basicamente o que estamos fazendo é criando arrays de arrays!

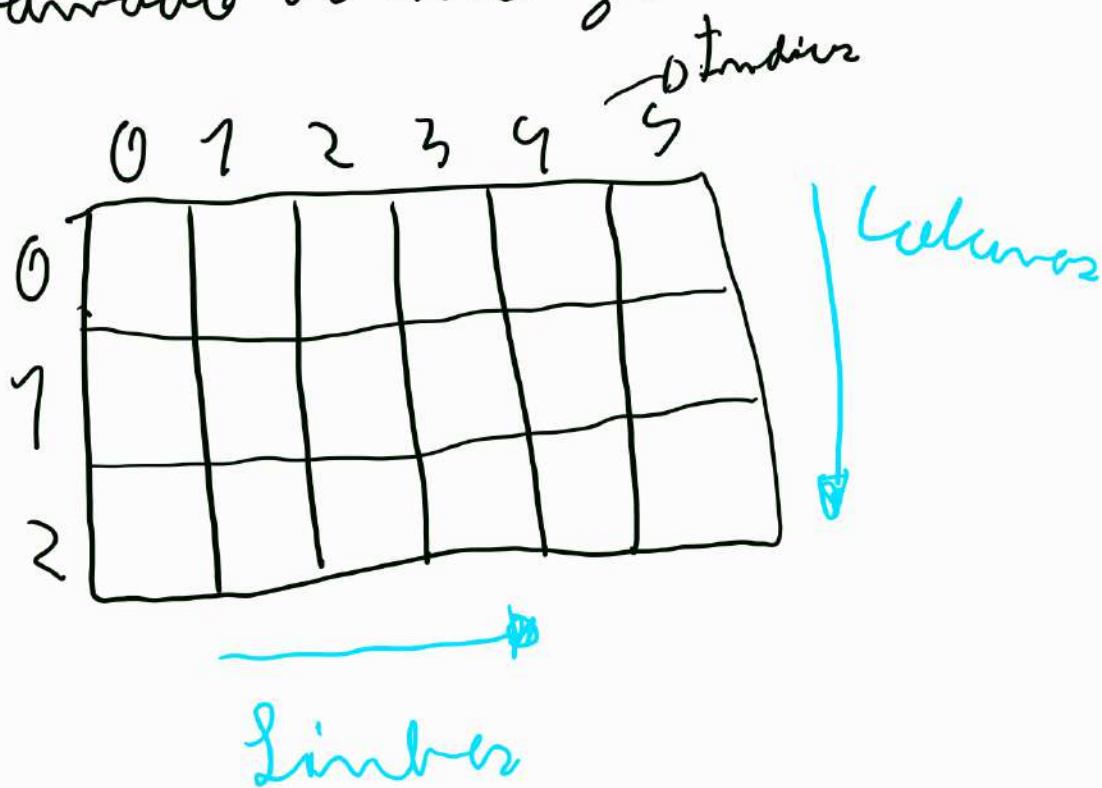
* Da mesma maneira que os arrays unidimensionais, os multidimensionais devem ser criados!

= Criando o multidimensional:

[tipo nome[][] = novo tipo[tamanho][tamanho];

↳ Criando e declarando de uma única vez!

= Um array de duas dimensões comumente é chamado de matriz!



nome[][]

↑ ↑
Linhas Colunas

↳ O primeiro índice representa as linhas, já o segundo as colunas!

• Lista:

São coleções que mantêm seus elementos em ordem e permitem elementos duplicados!

- Insere elemento na lista de acordo com o objeto! Ela não aceita tipos primitivos!

- Na lista usamos o método `get(indice)` para navegarmos pela lista! `lista.get(x)`; `get` sempre retorna um objeto! O `get` retorna o tipo objeto! Quando o `get` retorna não começamos usar nenhum método dele pois não temos a referência dele! É sim a referência da lista! Para a lista achar o objeto!

precisamos pegar a referência
para o objeto para aí nós podemos
criar os métodos de objeto!

- Para criarmos uma lista
podemos criar o `for each`!
Quando queremos percorrer sempre
irá percorrer a lista toda!

- Quando recebemos o objeto
de um `arraylist` sem um tipo específico
precisamos usar o `abstract`
para garantir para o compilador
que ele estará recebendo um `Objeto X`.

```

public class TesteArrayList {
    public void static main( String args [] ) {
        ArrayList<Integer> li = new ArrayList<Integer>();
        li.add( new Integer(1) );
        li.add( 2 );
        for (int i=0; i < li.size(); i++) {
            int n = li.get(i);
            System.out.println( n );
        }
    }
}

```

• Definindo o tipo de dado que a lista terá! (OBS: Integer é um objeto com o tipo primitivo int e não o primitivo em si!) Listas não armazenam objetos!

- Listas podem ser ordenadas da maneira que desejarmos! Lista é um dos tipos de estrutura de dados! Podemos utilizar uma série de algoritmos com ela!

• Hashmap:

- Um tipo de estrutura onde
o dado não é sempre armazenado
de forma "aleatória", porém todo
o conjunto de dados tem uma "key"!

A primeira declaração do
tipo é sempre a que será a key!

HashMap < Tipo, Tipo >

o outro tipo
será o conjunto dos dados!
(o tipo deve ser sempre um objeto)

HashMap:

Key + Dados	Key + Dados	Key + Dados
---	-------------	-------------

↳ Lembrete: As key podem ser
de qualquer tipo! Assim como os
dados armazenados no conjunto!

```
HashMap<String, String> loginESenha = new HashMap<>();
```

Tipo da key
do Hash

Os dados

Arrumando os
listas (e tudo na mão) em
HashMap também é um objeto
e deve ser instanciado!

* Lembrar: Arrumando nas listas
os HashMaps só armazenam objetos!
eles não armazenam tipos primitivos! Então
sempre precisamos declarar o objeto!

* Lembrar: O HashMap armazena
elementos de 2 objetos o primeiro
sempre será a key e o segundo
os dados!

• Exceções (Exceptions):

- Durante a execução de um programa várias situações não previstas de gerar erros!



■ As operações podem gerar diferentes tipos de erros devido a:

Ambiente de execução

✓ Ex: Acesso a arquivo, acesso à rede, falta de memória

Erros de programação

✓ Ex: Dividir por zero, acessar uma array fora dos limites de seu tamanho

Condições de negócio do programa

✓ Ex: Retirar dinheiro de uma conta sem saldo, diminuir o salário de um funcionário, atribuir uma nota inválida a um aluno.

- Exceção em Java é um sinal que indica a ocorrência de alguma "condição excepcional"

• O compilador Java obriga o usuário da classe a tratar as possíveis erros que um método pode gerar!

- Criando uma exceção específica:
- Por padrão o Java já vem com a classe "Exception" que é o "pai" de todas as exceções!

• Criando um novo tipo de exceção:

Exemplo:

```
public class NotaInvalidaException extends  
Exception  
{ }
```

• Todas as novas exceções devem "extends Exception" da classe pai das exceções!

• A nova exceção criada deve sempre seguir um padrão de ter o final de seu nome a palavra "Exception".

- Tratando as exceções:

- Podemos tratar as exceções de duas maneiras:

1º Dentro do método que gera a exceção.

2º Dentro do método que chama o método que gera a exceção.

1º: Quando tratarmos dentro do método que gera a exceção não decharamos "throw" (lançar) em sua assinatura, pois ele mesmo já está recebendo!

2º: Quando outro método está recebendo a exceção de outro, então precisamos declarar "throw" (lançar) em sua assinatura!

• Bloco try / catch / finally

- O bloco **try-catch** tem por finalidade sinalizar onde podem ocorrer exceções (**try**) e onde as exceções podem ser capturadas (**catch**), caso alguma exceção ocorra

```
void processarNota() {  
    try {  
        Estudante e = new Estudante();  
        e.atribuirNota(1, 11.0);  
    }  
    catch (NotaInvalidaException err) {  
        System.out.println("Erro na  
        atribuição de nota!");  
    }  
}
```

Pode lançar uma exceção

*Caso ocorra a exceção
X então y.*

• Bloco finally:

- O bloco **try-catch** possui ainda uma terceira parte opcional chamada **finally**
- O bloco **finally**, quando existir, **sempre** será executado, ocorrendo ou não uma exceção
- *Devemos tomar cuidado ao usar "return" em um finally.*
- O bloco **finally** é geralmente usado para fechar conexões com banco de dados, resetar variáveis, enfim, ações que devem ser tomadas caso erros ocorram ou não

- Exemplo:

```
try {  
    Estudante e = new Estudante();  
    e.atribuirNota(1, 11.0);  
} catch (NotaInvalidaException e) {  
    System.out.println("Erro na atribuição nota!");  
} finally {  
    System.out.println("Sempre passa aqui!");  
}
```

Sempre é executado!

Sempre: Um método pode lançar
várias exceções, desde que estejam declaradas
na sua assinatura!

```
try {  
    Estudante e = new Estudante();  
    e.atribuirNota(1, 11.0);  
}  
catch (NotaInvalidaException e) {  
    System.out.println("Erro na atribuição de nota!");  
}  
catch (Exception e) {  
    System.out.println("Erro desconhecido!");  
}  
  
finally {  
    System.out.println("Sempre passa aqui!");  
}
```

Dois blocos
catches, a
mais
específica
vem primeiro

Podemos ter vários blocos catch que funcionam
como uma "espécie de switch" onde dependendo da
exceção que for lançada será feita uma
tratativa diferente!

• Clases Excepciones:

- Assim como uma classe comum, as exceções podem armazenar valores que ajudam a compreender o erro! Podem também ter seus próprios métodos!

```
314 public class SemPedrasException extends RuntimeException {
315
316     public SemPedrasException(String mensagem) {
317         super(mensagem);
318     }
319     // Uma o construtor de Exception para
320     // notificar a classe do erro!
321 }
322
323 public class SemEncaixeException extends RuntimeException {
324     private int pontaA;
325     private int pontaB;
326
327     // Armazena os valores que deram errado!
328     public SemEncaixeException(int pontaA, int pontaB) {
329         super("Sem encaixe para a pedra jogada!");
330         this.pontaA = pontaA;
331         this.pontaB = pontaB;
332         // Construtor para armazenar
333         // os valores e a forma ao criar
334         // uma exceção!
335     }
336
337     public int getPontaA() {
338         return pontaA;
339     }
340
341     // Retorna o valor que deu errado
342     // da ponta A!
343
344     public int getPontaB() {
345         return pontaB;
346     }
347
348     // Retorna o valor que deu errado
349     // da ponta B!
350 }
```

• Atribuições:

- Sempre teremos apenas a referência de onde os desejos estão na memória e não de fato o desejo! Teremos sempre a referência!!!

Arrebatada ≠ Simples

Arrebatada tudo ao mesmo tempo, sem espera na execução!

{ Arrebatada apenas uma execução por vez, mesmo que a requisição seja feita exatamente ao mesmo tempo,

{ apenas ocorrerá uma execução por vez!

- Lembrete:

Quando não declaramos a "proteção" de encapsulamento é porque está por padrão definido como **private**!

Ex: **Sem nada declarado = Private!**
↓
`void teste()`

• Mapeamento:

- HashMap: Mais utilizado no Java!

• Exceções em Java:

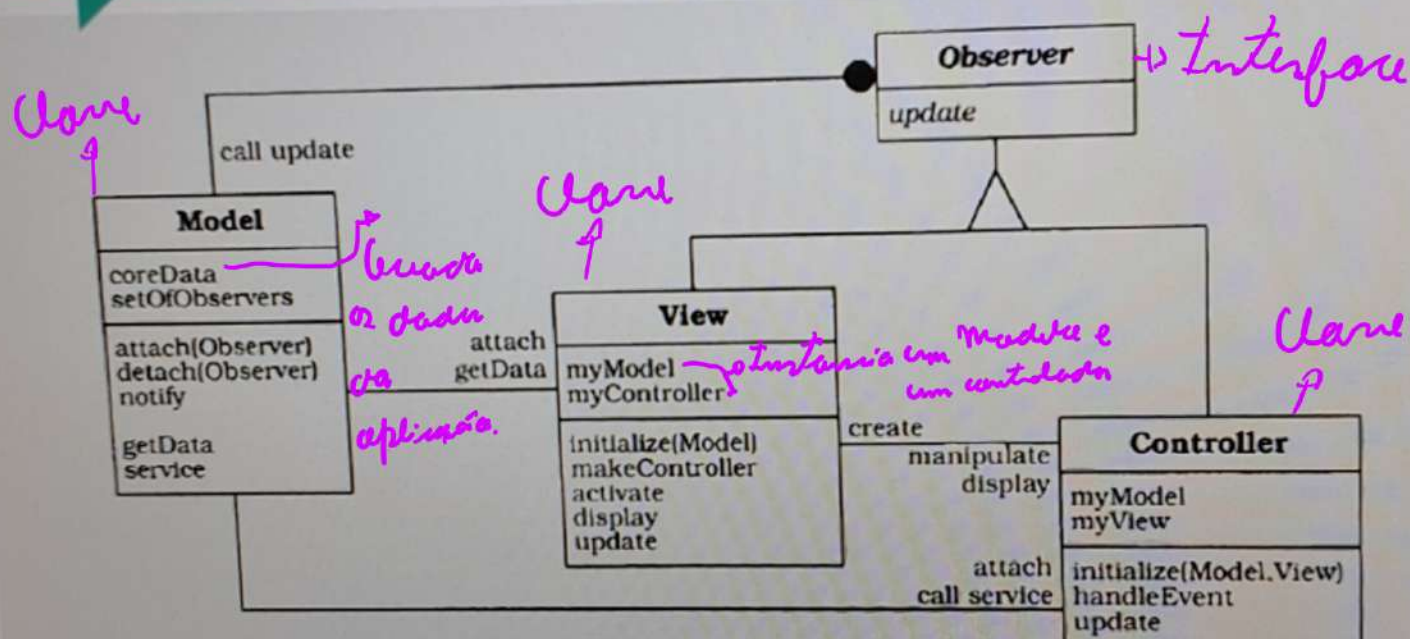
- É um sinal que indica a ocorrência de alguma condição.
- Em Java é possível criarmos nossa própria exceção:
 - Se for necessário criar diversos tipos uma classe exceção! Dessa maneira podemos criar centos de tipos que não se encaixam por meio de herança (Generalização).
- Uma exceção deve ser tratada, se não o programa é interrompido!
- Se criarmos um lançamento sem o catchador a coisa não funciona!
- Existem dois tipos de exceções:
 - **Exception** - Para erros verificáveis.
 - **RuntimeException** - Para erros não verificáveis.

- Iowa obriga a tratar os Exceptions,
pois não "prevêm-se" então obrigatoriamente
devem ser tratados!

- Perante Exceptions não a obrigato-
riedade de tratamento, pois acontecem
em tempo de exceção, em outra maioria
não erro de lógica! Não a obrigatoriedade
de tratar, porém se for encontrado a possibilidade
deve ser um tratado!

• Padrão de Arquitetural MVC

- Model - View - Controller
divide uma aplicação interativa
em três componentes!



* Quem cria o controlador é a View! É mais o contrário!
 Modelo: Dado e Funcionalidade!
 O modelo se trata e sabe de tudo!
 mesmo o controlador validando o modelo
 pode validar novamente para tornar
 certo!
 View = Onde mostra na tela!
 a view só mostra, ela não faz mais
 nada! A view não pode ser nenhuma outra
 classe.

Controlador - Verifica os eventos!
O controlador é um intermediário
entre o view e o modelo! O controlador
também pode fazer algumas coisas
como validação dos dados!

Main = O arquivo que terá o método
main para iniciar tudo!

- Lembrte: Em OO quando um
objeto chama um método dizemos
que ele está mandando uma
"mensagem" para um outro objeto.

• Manipulação de Strings:

- String em Java não é imutável, sempre que modificar uma string na verdade estará criando outra!

- `charAt()`: Pega um caractere pelo índice da string!

- `equals()`: Compara duas strings e retorna exatamente igual ou não.

- `substring()`: Separa parte de uma string através dos parâmetros índice!

- `trim()`: Remove espaços em branco finais e iniciais de uma string!

- `join()`: Junta textos, junta várias strings.

- `toUpperCase()`: Deixa todos os caracteres em maiúsculo.

- `toLowerCase()`: Deixa todos os caracteres em minúsculo.

- `toCharArray()`: Converte a `String` em um array de caracteres. `char[] string = "string";`

- `indexOf('buno')`: Retorna o index da busca se achar no texto, se não retorna -1.

- `matches()`: Método para verificar se uma `String` inteira bate com uma expressão regular (regex)

Regex é uma linguagem para descrever padrões de texto.

• Exemplo regex:

Símbolo	Significado	Exemplo
.	qualquer caractere	<code>a.b</code> → "acb", "aab"
<code>\d</code>	dígito (0-9)	<code>\d+</code>
<code>\w</code>	letra, número ou <code>_</code>	<code>\w+</code>
+	1 ou mais	<code>\d+</code>
*	0 ou mais	<code>a*</code>
?	0 ou 1	<code>a?</code>
[]	conjunto	<code>[aeiou]</code>
^	início da string	<code>^abc</code>
\$	fim da string	<code>abc\$</code>

• Listas encadeadas.

• Em Java, assim como em C, os objetos não se copiam literalmente, mas sim, fazem uma referência a memória (como em ponteiros)

$X = Y$

→ X está armazenando a referência do objeto Y! Se criar X.novo, estamos na verdade alterando o objeto Y!

Java não é feito criando um novo objeto e chamar "novo", assim como feito criando um objeto dentro da memória, não apenas uma referência a ele! → Este conceito é muito importante para podermos usar as listas encadeadas assim como na C.

Java - Referência | C - Ponteiros

• Modificadores:

- Static.

↳ Ex: static int número = 0;

→ O modificador static pertence a classe e não ao objeto! Desta maneira se atualizarmos o valor temos os objetos também sendo atualizados!

↳ Ex: static int número = 5;

→ Todos os objetos agora terão número como 5!

OBS: Para atualizar um atributo static devemos atualizar a classe e não o objeto!

- Final modifier:

- Pode ser usado na clare, no
método ou no atributo?

- Atributo - O valor não pode
ser alterado!

- Método: Não pode ser substituído.

- Clare: Não pode ser herdada!

RunTime polymorphism

VS

Compile-Time polymorphism

Compile:

- O que é escolhido é durante a compilação!
- Exemplo: recarga de métodos, pois tem assinaturas diferentes, então o compilador já sabe o que irá utilizar.

RunTime:

- O que é escolhido é durante a execução!
- Exemplo: reescrita de métodos, pois tem exatamente a mesma assinatura, então o compilador não sabe quem deve chamar durante a execução.

SOBRECARGA = MUDA ASSINATURA

≠

SOBRESCRITA = MESMA ASSINATURA

- Inner Class!
 - Um Java uma classe pode conter outra classe dentro!
 - Útil para comportamentos que não fazem sentido dentro de uma classe específica.

Uma inner class pode acessar atributos privados da classe externa.

Java

```
class Conta {  
  
    private double saldo = 1000;  
  
    class Verificador {  
        void mostrarSaldo() {  
            System.out.println(saldo);  
        }  
    }  
  
}
```

Relembra também como
static para mudar o
comportamento.

Mesmo sendo `private`, a inner class consegue acessar.

↳ Útil, mas deve se tomar muito cuidado para que o código não fique confuso, deve se usar apenas se não tiver um jeito melhor.

• Local Inner Class:

- O Java permite classes dentro
dos métodos!

3 Serve para classes temporárias

É usada quando:

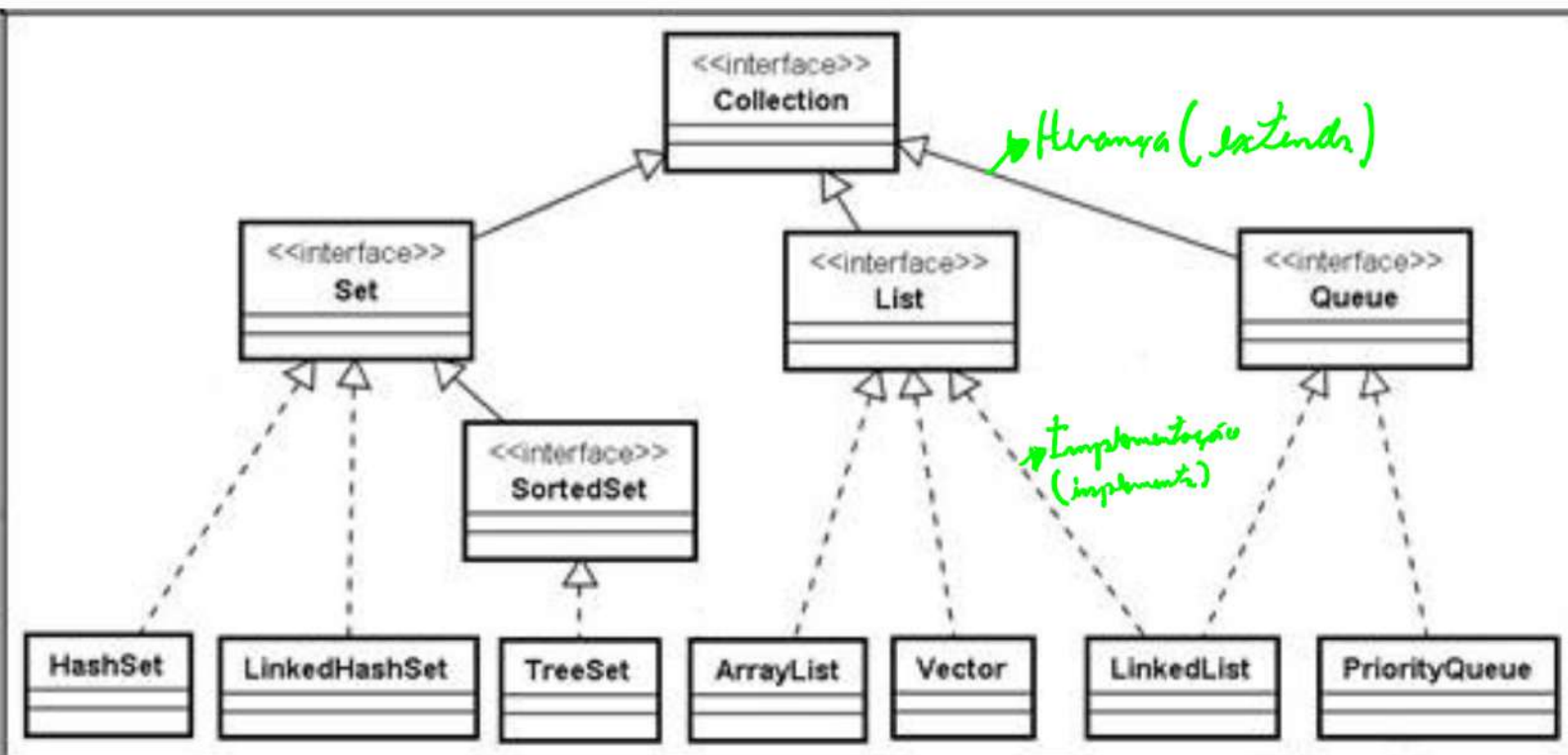
- A classe só é necessária naquele método
- Você não quer criar uma classe separada
- É algo muito específico daquele método

↳ Só foi sentido em cores tem
especificar, movimento de uma forma
muito cuidado pois podem
deixar o código muito difícil de
compreender!

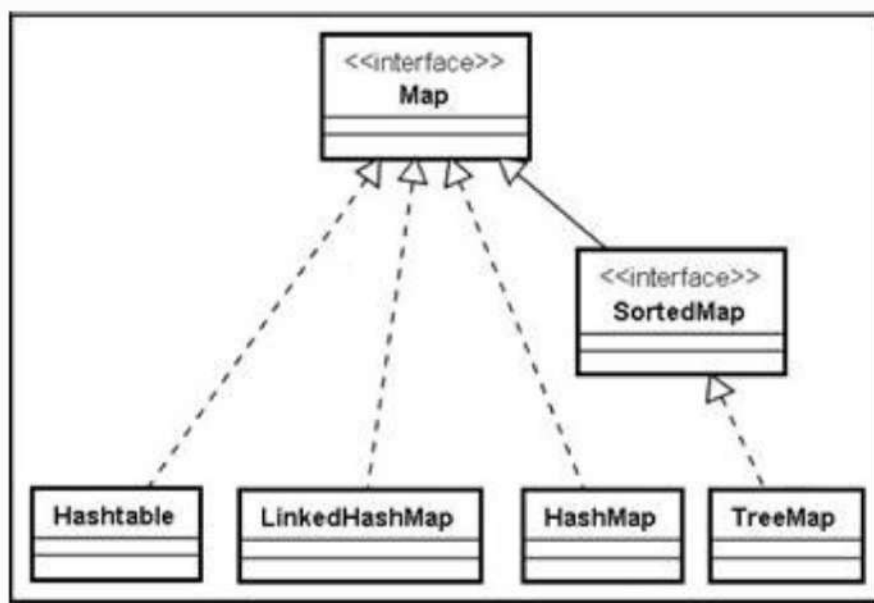
• Java Collections:

"Conjunto de interfaces e classes para representar e tratar grupos de dados"

- Árvore de hierarquia:



- Árvore de hierarquia dos MAPs: (não se trata de categorias, mas estão no mesmo nível)



• Erros não os dados que fazem parte do Java Collections:

ArrayList – Lista baseada em array que pode crescer dinamicamente. Possui **acesso rápido** aos elementos, porém **inserções e remoções são mais lentas (lineares)**. Ideal quando a prioridade é **acessar elementos rapidamente**.

LinkedList – Lista ligada onde cada elemento aponta para o próximo. **Inserções e remoções são rápidas**, mas a **busca é linear**. Melhor quando há muitas **inserções e exclusões**.

HashSet – Conjunto que **não permite elementos duplicados** e **não garante ordem**. Possui **acesso rápido**. Indicado quando a **ordem não importa** e os elementos devem ser **únicos**.

TreeSet – Conjunto que **mantém os elementos ordenados automaticamente**. Porém o **acesso é mais lento que no HashSet**. Use quando precisar de **elementos únicos em ordem natural**.

LinkedHashSet – Variante do HashSet que **mantém a ordem de inserção** dos elementos. Útil quando é necessário **elementos únicos e preservar a ordem em que foram adicionados**.

HashMap – Estrutura de **pares chave-valor**, baseada em tabela hash. **Permite chaves e valores null** e **não garante ordem**. Boa quando se precisa **associar um identificador (chave) a um valor com acesso rápido**.

TreeMap – Mapa que **mantém as chaves ordenadas automaticamente**. É **mais lento que HashMap**, mas **garante ordem crescente (ou personalizada)**.

LinkedHashMap – Mapa que **mantém a ordem de inserção das chaves** usando lista duplamente ligada. Ideal quando é necessário **mapa chave-valor preservando a ordem de inserção**.

↳ Implementar a estrutura de dados e seus respectivos algoritmos!

- Se não usarmos essas classes diferentes implementamos manualmente! (Arrumamos tudo um por um e cada um com suas próprias implementações dependendo de cada caso!)

Java Collections não são thread-safe.

- As estruturas permitem apenas inserir objetos e também null.

- Collections tem mais de 40 métodos utilitários (depende da versão do Java) para poder ordenar, buscar, inserir, remover, verificar e manipular coleções.

• Anúncios:

- Ideal para estruturas FIFO (Fila).

• File and Directory the Java File Handling

- `import java.io.*`

↳ Importamos os classes que precisamos para manipular os arquivos em Java.

• Diferenças entre os classes Files:

File class: Comparison

Class	Usage	Pros	Cons
File	Represents a file/directory	Simple representation	Does not perform file operations
FileReader	Reading character streams	Easy to use for text files	Cannot read binary files
BufferedReader	Efficient reading of text	Reads large files efficiently	Requires wrapping around FileReader
FileWriter	Writing character streams	Simple for text files	Cannot write binary files
BufferedWriter	Efficient writing of text	Writes large files efficiently	Requires wrapping around FileWriter

• Byte streams: Applications:

- Li é uma organização de Bytes!

- Usa a mesma biblioteca Java.io.

- Bom para Images, videos, network programming, serialização de objetos.

Byte streams vs. character streams

Feature	Byte Streams	Character Streams
Data Type	Raw binary data	Character data (text)
Base Classes	InputStream, OutputStream	Reader, Writer
Common Subclasses	FileInputStream, FileOutputStream	FileReader, FileWriter
Encoding	Handles raw bytes	Uses character encoding
Usage	For binary files (images, audio)	For text files

• Criando a File class:

```
File myFile = new File("example.txt");
```

↓
type

↓
nome

↓
Constructor armazena o caminho até a pasta!

O objeto File armazena somente o caminho até a pasta e não a pasta toda! Só o caminho!

```
if (myFile.exists())  
{
```

```
    System.out.println("File exists.");
```

```
    } else {
```

```
        System.out.println("File does not exist.");
```

- O método `exists()` irá verificar se de fato a pasta `exists()` de acordo com o caminho armazenado no objeto `myFile`.
Se sim, true, se não, false.

- Código completo:

```
import java.io.File;

public class
FileExample {
    public static void main
    (String[] args) {
        File myFile = new File("example.txt");
        if (myFile.exists())
        {
            System.out.println("File exists.");
        } else {
            System.out.println("File does not
            exist.");
        }
    }
}
```

↳ Ao final imprime se de fato o arquivo existe ou não existe!

- Difer: Para apagar um arquivo de uma pasta;

• Java NIO:

- Oferece uma melhor abordagem para lidar com diretórios em comparação com o java.io tradicional.

- import java.nio.*

• java.time:

Class	Purpose	Time Zone Awareness	Example Use Case
LocalDate <i>↳ só a data (Não precisa da hora)</i>	Represented dates in year, month, day	No	Storing birthdays
LocalTime <i>↳ só o horário (Não precisa da data)</i>	Represents local time in hours, minutes, seconds	No	Setting alarm clocks
LocalDateTime <i>↳ Data + Hora (Sem fuso)</i>	Represents both date and time without a timezone	No	Scheduling meetings
ZonedDateTime <i>↳ Data + Hora + Fuso.</i>	Represents date-time with timezone	Yes	Handling international flight times

- import java.time.LocalDate;

- import java.time.*...

↳ Cada classe irá conter seus métodos para lidar com data e hora.
Exemplo: now()

• Formatting dates:

- Banco de dados ou infopos de créancieros muitas das vezes aceitam dados de uma maneira específica, por isso é necessário formatar!

import java.time.format.DateTimeFormatter;

↳ Alguns diversos tipos de formatação para dados através da classe DateTimeFormatter.

- Fuso horário:

- UTC (Coordinated Universal Time):

- Relógio global oficial.

- Todos os fusos horários do mundo são calculados a partir de UTC.

- Exemplos:

- Brasil: Geralmente UTC-3

- Londres: UTC+0

- Tóquio: UTC+9

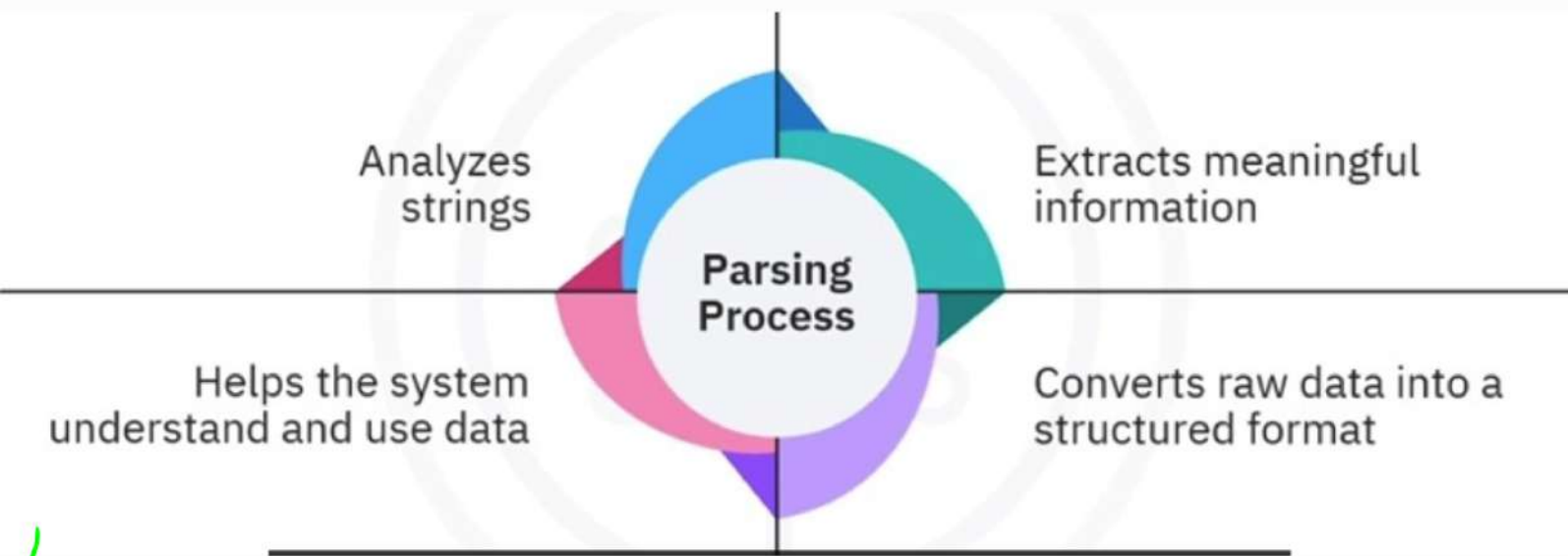
- Classes que lidam com fuso horário:

Time zone: Classes



• Parsing:

- Process de análise.



↳ Ciclo de processo de uma análise.

Processo onde analisamos um dado x
(normalmente uma string) e convertemos
para um formato que o usuário e o
sistema entendam.

